

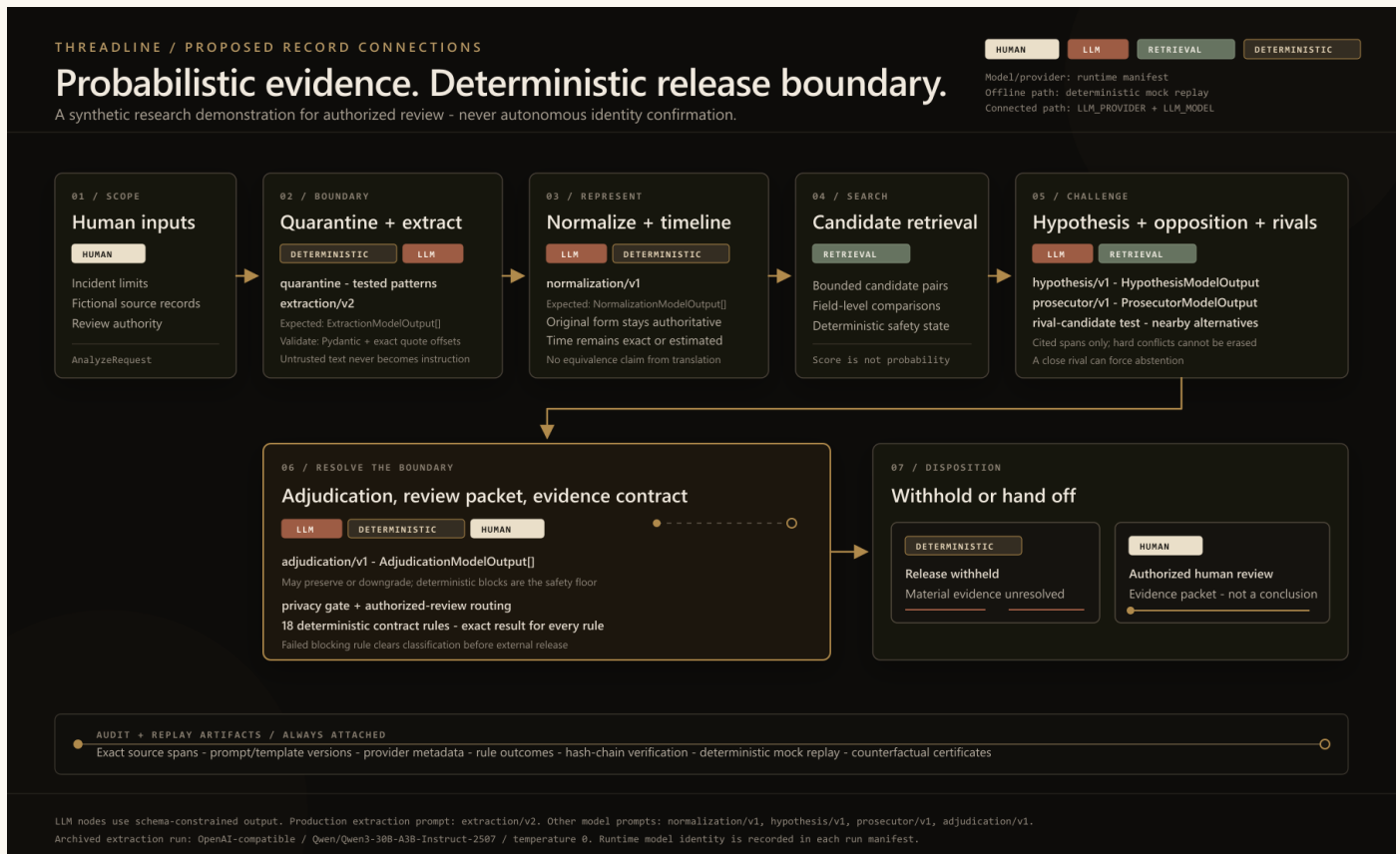
THREADLINE / WORKFLOW REFERENCE

Evidence in. Boundaries out.

A printable description of all twelve workflow nodes, five connected-provider LLM stages, deterministic validation boundaries, release rules, failure behavior, and known limitations.

SCOPE

Possible record connections for authorized human review. The system preserves evidence and uncertainty; it never confirms identity.



Editable source: [threadline-workflow.svg](#). The privacy gate is intentionally shown after the five model stages because the current implementation protects the reviewer-facing packet, not provider ingress.

THREADLINE / MODEL AND PROVIDER BOUNDARY

Version: threadline-workflow/2.0.0 Scope: synthetic research demonstration Decision boundary: proposed record connections for authorized human review only

THREADLINE is an experimental safety and review layer for proposed record connections. It preserves exact provenance, challenges each hypothesis with contradictions and rival candidates, and deterministically withholds outputs that are not safe for authorized human review.

The runtime executes twelve ordered nodes. A deterministic evidence contract then controls external release. None of these stages confirms identity, declares a person found, or merges records.

Model and provider boundary

Five nodes use schema-constrained model output: extraction, multilingual normalization, hypothesis, contradiction challenge, and adjudication. Provider and model are selected at runtime through LLM_PROVIDER and LLM_MODEL and recorded in run metadata.

- Offline judge path: deterministic mock provider; no credentials or network required.
- Connected path: configured OpenAI-compatible provider; provider behavior may be nondeterministic.
- Promoted extraction prompt: extraction/v2.
- Other model prompts: normalization/v1, hypothesis/v1, prosecutor/v1, and adjudication/v1.
- Archived Prompt V2 extraction evaluation: OpenAI-compatible provider, Qwen/Qwen3-30B-A3B-Instruct-2507, temperature 0.

This archive evaluates extraction plus downstream deterministic policy; it is not evidence that every model-backed workflow node was run live under that model.

LLM stage	Prompt	Expected output	Deterministic boundary after output
Structured extraction	extraction/v2	ExtractionModelOutput[]	Schema, field allowlist, and exact quote/offset validation
Multilingual normalization	normalization/v1	NormalizationModelOutput[]	Original forms and source links must remain intact
Evidence hypothesis	hypothesis/v1	HypothesisModelOutput	Every support factor must cite an admissible source span
Contradiction challenge	prosecutor/v1	ProsecutorModelOutput	Missing deterministic blockers are restored
Independent adjudication	adjudication/v1	AdjudicationModelOutput[]	Transition guard permits preservation or downgrade only

The privacy gate occurs after these five model-backed stages. It minimizes the reviewer-facing packet; it is not a provider-ingress privacy control.

Node summary

Order	Node ID	Type	Input	Output	Prompt
1	incident	Human input + deterministic validation	AnalyzeRequest	IncidentConfiguration Output	None
2	quarantine	Deterministic safety boundary	RecordInput[]	QuarantinedRecord[]	None
3	extract	LLM transformation	QuarantinedRecord[]	ExtractionModelOutput[]	extraction/v2
4	normalize	LLM transformation + deterministic helpers	ExtractionModelOutput[]	NormalizationModelOutput[]	normalization/v1
5	timeline	Deterministic validation	NormalizedRecord[]	TimelineEvent[]	None
6	retrieve	Retrieval + deterministic comparison	NormalizedRecord[]	CandidateConnection[]	None
7	hypothesis	LLM reasoning	CandidateEvidence	HypothesisModelOutput	hypothesis/v1
8	prosecutor	LLM reasoning	CandidateEvidence	ProsecutorModelOutput	prosecutor/v1
9	rivals	Retrieval	CandidateConnection[]	RivalComparison[]	None
10	adjudicate	LLM reasoning + deterministic transition guard	AdjudicationPacket	AdjudicationModelOutput[]	adjudication/v1
11	privacy	Deterministic reviewer minimization	CandidateReviewPacket	PrivacySafePacket	None
12	review	Human-review routing	PrivacySafePacket	HumanReviewRoute	None

01 - Incident configuration

- Purpose: Validate incident scope, fictional-data status, request size, candidate limits, and the review boundary.
- Tool: Deterministic Python and Pydantic validation with human-supplied incident configuration.
- Validation: Record count, per-record text length, total text length, and candidate limits must pass configured bounds.
- Failure or abstention: Rejects the request when metadata or safety limits are invalid.
- Why separate: Prevents an oversized or out-of-scope request from entering model-backed stages.
- Known limitation: The code validates declared scope; it does not independently prove that submitted data is fictional or that a user is authorized.

02 - Evidence quarantine

- Purpose: Treat every source narrative as untrusted evidence and isolate tested instruction-like patterns.
- Tool: Deterministic pattern detection and safe-evidence rendering.
- Validation: Original source text must remain byte-for-byte unchanged; record content never enters a system message.
- Failure or abstention: Fails if a downstream-safe representation cannot be produced while preserving the original.
- Why separate: Establishes the prompt-injection boundary before any model call.
- Known limitation: Pattern-based quarantine is not universal prompt-injection protection. The original and safe representations remain available for audit.

03 - Structured extraction

- Purpose: Extract typed fields tied to exact original source spans.
- Model/tool: Runtime model provider or deterministic mock; prompt extraction/v2; expected schema `ExtractionModelOutput[]`.
- Deterministic validation: Pydantic schema validation, canonical field-key normalization, exact half-open quote/offset validation, and one evidence-correction retry. Unsupported fields are removed or marked missing.
- Failure or abstention: A non-missing field that cannot retain an exact source span is withheld from downstream support.
- Why separate: The model converts narrative to structure; deterministic code decides whether the structure is evidentially admissible.
- Known limitation: Extraction can miss a real fact, attach a field incorrectly, or inherit provider variability. One archived live run is not a repeatability study.

04 - Multilingual normalization

- Purpose: Add comparison representations without overwriting native-script or source-language evidence.
- Model/tool: Runtime model provider or deterministic mock; prompt normalization/v1; deterministic Unicode, transliteration, and name-normalization helpers.
- Deterministic validation: The model output must preserve the original form and its evidence links. Candidate variants remain aids, not equivalence claims.
- Failure or abstention: A representation is not accepted if it changes the original form or loses lineage.
- Why separate: Keeps semantic representation distinct from source extraction and from any conclusion about a person.
- Known limitation: Transliteration and translation can lose meaning. Original-language review remains authoritative.

05 - Timeline reconstruction

- Purpose: Order time and location evidence while preserving exact versus estimated status.
- Tool: Deterministic timestamp parsing and chronology rules.
- Validation: No timeline event can be marked as identity proof; approximate times remain approximate.
- Failure or abstention: Withholds a coherent chronology when it would require converting uncertainty into fact.
- Why separate: Makes chronology inspectable before it is used as support or contradiction.
- Known limitation: Plausible travel does not prove a connection, and coarse location data can hide a real conflict.

06 - Candidate retrieval and comparison

- Purpose: Generate bounded candidate pairs, compare fields, and calculate a deterministic safety state.
- Tool: Deterministic blocking rules, candidate generation, pairwise comparison, scoring, and linkage policy. No model call occurs here.
- Validation: Unique-identifier candidates survive caps; missing values do not become contradictions; scores are ranking signals, not probabilities.
- Failure or abstention: Fewer than two records or no discriminating fields produces explicit insufficient evidence. A documented legacy fallback can retain recall when no blocking strategy fires.
- Why separate: Separates search recall from the later hypothesis, challenge, and release decisions.
- Known limitation: Candidate caps and blocking rules can miss a plausible record. Retrieval recall must be measured separately from released-proposal recall.

07 - Evidence hypothesis

- Purpose: Summarize the strongest supporting evidence already produced by deterministic comparison.
- Model/tool: Runtime model provider or deterministic mock; prompt hypothesis/v1; expected schema HypothesisModelOutput.
- Deterministic validation: Every supporting factor must cite a valid source span. Unsupported factors are removed. A blocked or insufficient deterministic state cannot be upgraded.
- Failure or abstention: Unsupported support is removed and recorded; the node cannot invent a replacement.
- Why separate: Keeps the affirmative case isolated from the contradiction challenge.
- Known limitation: A concise structured hypothesis is not private chain-of-thought and may omit relevant context.

08 - Contradiction challenge

- Purpose: Search independently for evidence against each proposed connection.
- Model/tool: Runtime model provider or deterministic mock; prompt prosecutor/v1; expected schema ProsecutorModelOutput.
- Deterministic validation: Absence is not contradiction; hard and soft conflicts remain separate; deterministic blocking conflicts are restored if model output omits them.
- Failure or abstention: Unsupported opposing evidence is not promoted. Unresolved material conflict continues to the release gate.
- Why separate: Prevents supporting evidence from silently absorbing or softening opposition.
- Known limitation: The challenger can miss contradictions that were never extracted or represented.

09 - Rival-candidate test

- Purpose: Test whether the evidence distinguishes the leading candidate from nearby alternatives.
- Tool: Deterministic retrieval and field-consistent rival comparison.
- Validation: Equal or close rivals remain visible; a close rival prevents the strongest release state unless a qualifying unique identifier resolves it.
- Failure or abstention: Nearby candidates that cannot be distinguished can force insufficient evidence.
- Why separate: A leading candidate is not meaningful without testing plausible alternatives.
- Known limitation: The test can only examine rivals that retrieval found.

10 - Independent adjudication

- Purpose: Aggregate isolated support, opposition, rivals, and deterministic checks without weakening the safety floor.
- Model/tool: Runtime model provider or deterministic mock; prompt adjudication/v1; expected schema `AdjudicationModelOutput[]`.
- Deterministic validation: An explicit transition allowlist permits preservation or downgrade only. Blocked or insufficient states cannot become a stronger proposal.
- Failure or abstention: Missing inputs or an invalid transition leaves the safer deterministic state in place.
- Why separate: Tests whether structured reasoning changes the review disposition while preventing a model from overruling policy.
- Known limitation: Multiple model calls do not create independent evidence; they are structured reviews of the same packet.

11 - Privacy gate

- Purpose: Apply least-necessary redaction to reviewer-facing text while retaining the original evidence for audit.
- Tool: Deterministic pattern redaction.
- Validation: Redactions affect `safe_text`; original text remains preserved and each redaction is audited.
- Failure or abstention: Fails when a configured restricted field remains in reviewer-facing content.
- Why separate: Makes reviewer minimization explicit and testable.
- Known limitation: This node runs after model-backed stages. It is not provider-side data minimization, authentication, encryption, or a retention policy.

12 - Human-review router

- Purpose: Build an authorized-review packet with priorities, exact reason codes, unresolved issues, rivals, and permitted actions.
- Tool: Deterministic routing from the final structured state; actual disposition belongs to an authorized person.
- Validation: The packet exposes no autonomous identity-confirmation action and preserves abstention.
- Failure or abstention: A packet is invalid if it implies autonomous determination or bypasses authorized review.
- Why separate: Distinguishes system preparation from accountable human action.
- Known limitation: The demonstration has no production authentication, role enforcement, reviewer training, escalation service, or organizational governance.

Post-workflow release gate - evidence contract

After the twelve nodes, each candidate receives an 18-rule deterministic evidence contract. The contract checks exact source spans, lineage, certainty preservation, hard-conflict inclusion, rival coverage, safety language, audit integrity, and release authorization. Every rule emits a typed outcome and remediation. Any blocking finding clears the externally releasable classification and marks the packet Withheld. Review-required findings retain only the information safe for authorized review.

Audit, replay, and counterfactual artifacts

Runs persist original inputs, node inputs and outputs, prompt/template identifiers, provider metadata, validation results, evidence contracts, audit events, and replay manifests in local SQLite storage. Deterministic mock runs can be replayed exactly. Counterfactual certificates describe only the tested evidence change and first changed node; they do not prove identity.

Known system-level limits

THREADLINE is not a production registry, a public search engine, facial recognition, or real-world humanitarian validation. The repository contains fictional identities only. Authentication, authorization, encryption at rest, retention/deletion policy, data residency, connected-provider replay, representative field evaluation, fairness analysis, and external security review remain future work.

Authoritative sources

Workflow details are derived from the repository's workflow registry, node implementations, prompts, schemas, evidence-contract service, audit service, and deterministic submission artifacts. This PDF is a presentation derivative; use the Markdown and machine-readable files for audit.